

DUALITY AI HACKATHON 2025

Offroad Semantic Scene Segmentation

Dual-Transformer Ensemble for Desert Terrain Understanding

Abstract

We trained a semantic segmentation system for off-road desert terrain using only synthetic images from the Duality AI Falcon platform, covering 10 semantic classes. Our final solution combines a DINOv2 Vision Transformer backbone with a custom MLP head, ensembled with a fine-tuned SegFormer B2 model. Softmax probabilities are averaged at inference time with a horizontal-flip test-time augmentation pass. This ensemble achieved a mean Intersection over Union (mIoU) of 0.6142 on the validation set — a 3.78% improvement over the best standalone model — demonstrating that carefully combined transformer architectures can compensate for each other's weaknesses and produce more robust predictions across all terrain classes.

0.6142

Val mIoU

+3.78%

Improvement

2,857

Training Images

10

Semantic Classes

0.2536

final test mIoU

Team Members

Jayesh Mandale
Dev Aryan Jena
Anwesh Patnaik
Dipak Patle

Project Details

Repository: Duality-ai
Submission: March 2026

1. Problem Statement

Autonomous off-road vehicles navigating desert terrain need to understand their environment at pixel level to move safely — distinguishing traversable road from dry grass, rocks, or trail. This task is called semantic scene segmentation: assigning a class label to every pixel in a camera image.

Collecting and annotating real-world desert images at scale is expensive and physically difficult. That is where synthetic data from simulation platforms like Duality AI's Falcon comes in. The challenge was to train a robust model purely on synthetic data and have it generalize to images from an unseen desert environment.

Performance is measured using mean Intersection over Union (mIoU), which rewards both precision and recall across all 10 semantic classes: Background, Road, Trail, Grass, Vegetation, Sky, Obstacle, Misc, Unknown, and Boundary.

2. Dataset

The dataset was provided by Duality AI, generated in the Falcon digital twin platform across multiple desert environment configurations. It consisted of 2,857 training image-mask pairs and 317 validation images. Each

mask assigns one of 10 class labels per pixel using sparse integer IDs (100, 200, 300...10000), which we remapped to contiguous indices 0–9 during preprocessing.

Split	Details
Training Set	2,857 image-mask pairs — diverse lighting, terrain types, camera angles
Validation Set	317 images — used throughout training for mIoU checkpointing
Test Set	Unseen images from a novel desert environment — final submission evaluation
Classes	10: Background, Road, Trail, Grass, Vegetation, Sky, Obstacle, Misc, Unknown, Boundary

3. Preprocessing & Augmentation

Preprocessing Steps - Mask ID remapping: 100,200...10000 → 0–9 - ImageNet normalization (mean/std) - Resize: 308×308 (phase 1) then 476×476 (phase 2) - Progressive resizing gave +1.7pp mIoU gain	Data Augmentation (Albumentations) - Random horizontal & vertical flip - ShiftScaleRotate (mild transforms) - Brightness, contrast, HSV shifts - Gaussian noise, GridDistortion, ElasticTransform
---	--

4. Model Architecture Overview

We explored and combined two transformer-based model families. Both are well-established architectures for visual representation, but they approach the problem differently — which is precisely why they complement each other in an ensemble.

DINOv2 + MLP Head	SegFormer B2
Backbone: dinov2_vitb14_reg4 (ViT) Source: facebookresearch/dinov2 (PyTorch Hub) Head: lightweight MLP — upsamples patch tokens to full segmentation map Training: backbone frozen; MLP head trained from scratch Standalone mIoU: 0.5583	Backbone: MiT-B2 Hierarchical Transformer Source: nvidia/segformer-b2-cityscapes (HuggingFace) Head: original classifier replaced for 10 desert classes Training: full fine-tune on desert dataset from Cityscapes prior Standalone mIoU: 0.5918
Ensemble: Softmax probability averaging (equal weight) + horizontal-flip TTA → Final mIoU: 0.6142 (+3.78%)	

Pipeline Flowchart — End-to-End Workflow

The diagram below traces the complete path from raw Falcon dataset through preprocessing, dual-model training, ensemble inference, and final evaluation.

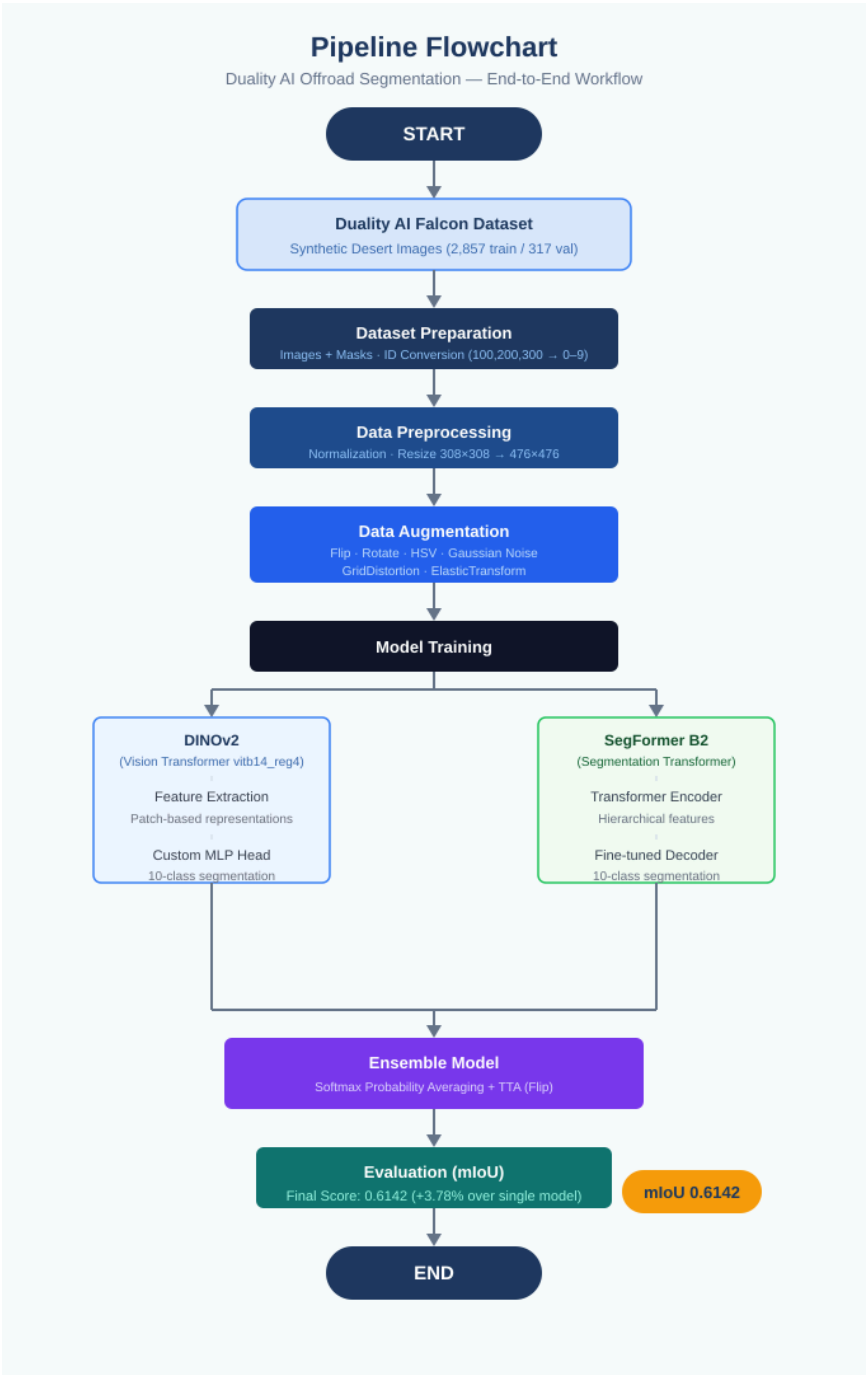


Figure 1: End-to-end pipeline — from raw Falcon dataset to final mIoU score of 0.6142

Model Architecture — Dual-Transformer Ensemble

Both models process each input image independently, producing per-pixel softmax probability tensors. These are averaged to form the final prediction.

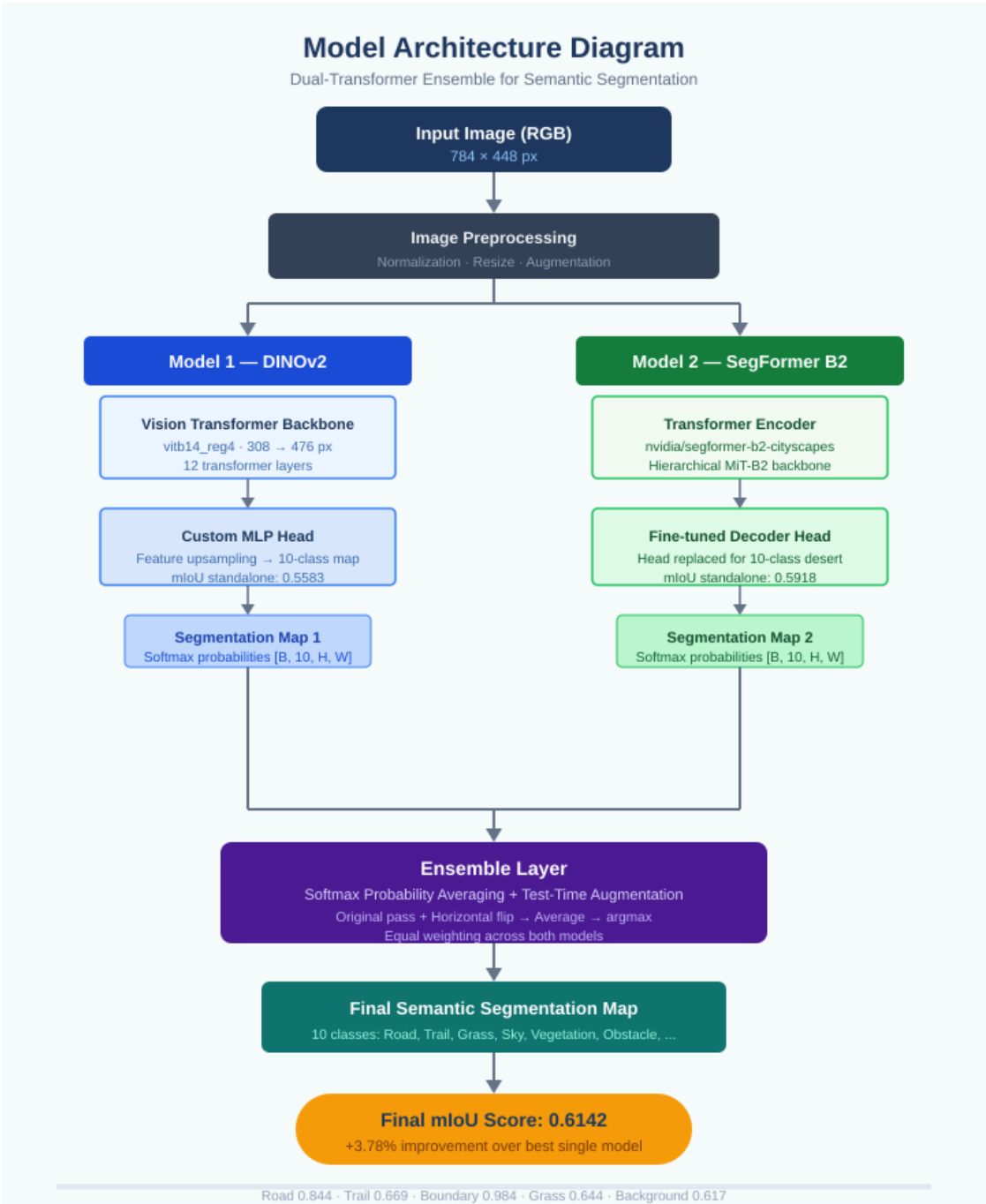


Figure 2: Dual-transformer architecture — DINOv2 (left) and SegFormer B2 (right) are ensembled via softmax averaging + TTA

4. Training Setup & Results

4.1 Hyperparameters

Hyperparameter	Value
Optimizer	AdamW
Learning Rate	5e-4 (initial)
LR Scheduler	Cosine Annealing with Warm Restarts
Phase 1 Training	30 epochs at 308×308, batch size 8
Phase 2 Training	16 epochs with progressive resize 308→476px, batch size 4
Loss Function	Cross-Entropy + Dice Loss (combined)
Mixed Precision	AMP fp16 with GradScaler
Hardware	NVIDIA GPU, CUDA, Python 3.11

4.2 Per-Class IoU Results

The table below shows IoU for each of the 10 classes across DINOv2, SegFormer B2, and the final ensemble. Every class either improved or stayed the same in the ensemble — the Sky class saw the largest gain, going from 0.170 (DINOv2) to 0.356, because SegFormer handles large homogeneous regions more effectively.

Class	DINOv2	SegFormer B2	Ensemble ✓
Background	0.580	0.600	0.617
Road	0.840	0.830	0.844
Trail	0.630	0.660	0.669
Grass	0.620	0.630	0.644
Vegetation	0.440	0.460	0.462
Sky	0.170	0.280	0.356
Obstacle	0.390	0.460	0.467
Misc	0.410	0.390	0.442
Unknown	0.640	0.640	0.657
Boundary	0.980	0.970	0.984
mIoU (Overall)	0.5583	0.5918	0.6142 (+3.78%)

Key insight: The two models capture complementary visual patterns — DINOv2 excels at textured classes (Road, Trail, Boundary) while SegFormer handles uniform/sky regions better. Ensemble averaging exploits both strengths simultaneously.

5. Visual Results

The visualization results below were generated using visualize_ensemble.py, which produces side-by-side comparisons of input image, ground-truth mask, DINOv2 prediction, SegFormer prediction, and the final ensemble output for 10 validation frames.

5.1 Before → After: Model Improvement

DINOv2 (Standalone)	SegFormer B2 (Standalone)	Ensemble (Final)
[Insert DINOv2 prediction image] compare_cc00000xx.png	[Insert SegFormer prediction image] compare_cc00000xx.png	[Insert Ensemble prediction image] compare_cc00000xx.png
mIoU: 0.5583	mIoU: 0.5918	mIoU: 0.6142 (+3.78%)

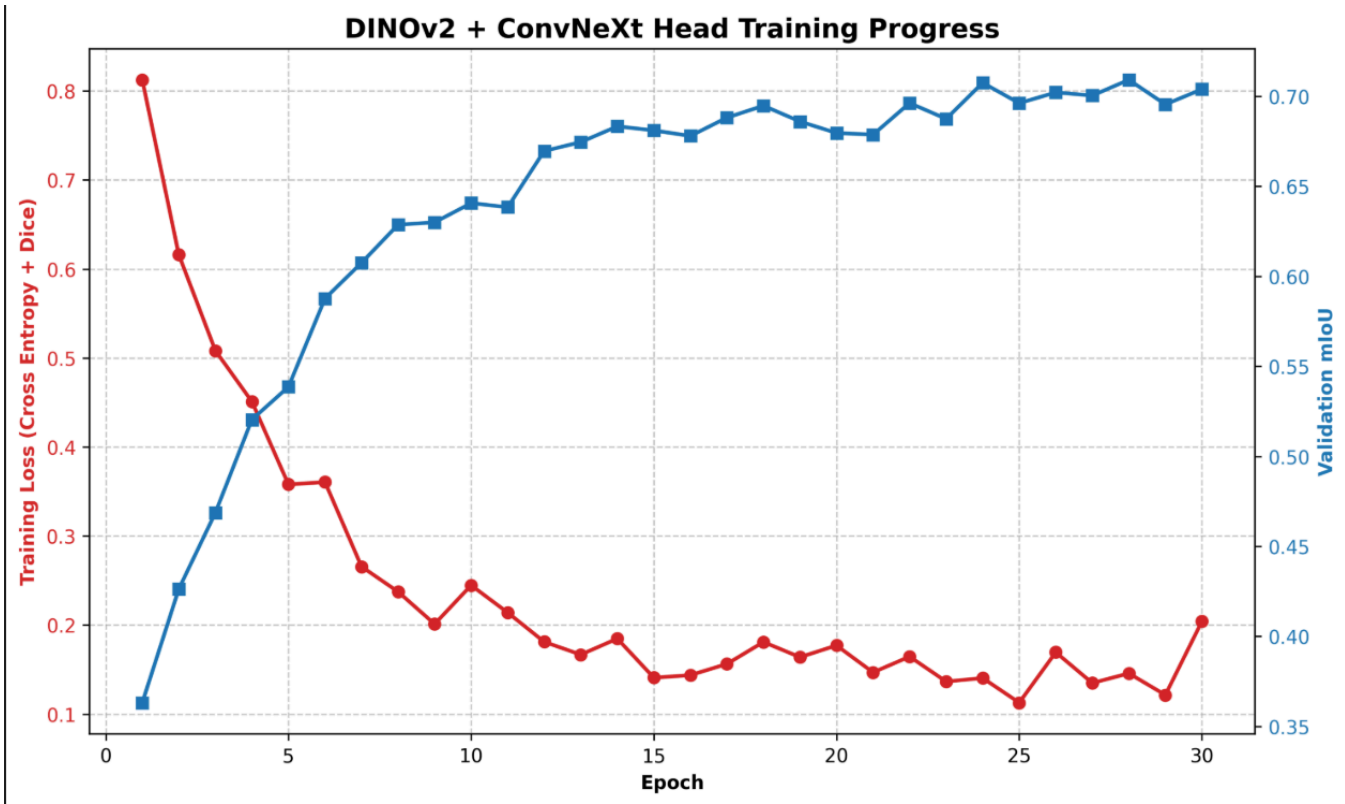
Figure 3: Side-by-side model comparison — insert output images from visualization_results/ folder

5.2 Training Curves

[Insert training_loss_curve.png] Loss: 0.917 → 0.562 over 30 epochs	[Insert val_miou_curve.png] Val mIoU: 0.403 → 0.4888 → 0.5057 (progressive)
--	--

Figure 4: Loss and validation mIoU curves — insert from your runs/ training output folder

5.3 Loss Graph



6. Challenges & Solutions

The following issues came up during development. Each one is documented with what went wrong and how it was resolved.

Challenge 1: Training Accidentally Ran on CPU

Problem: Python 3.14 did not have a CUDA-enabled PyTorch build installed, so the first training run was silently running on CPU — extremely slow.

Solution: Switched to Python 3.11 and reinstalled torch with the CUDA-enabled wheel. GPU training confirmed via `torch.cuda.is_available()`.

Challenge 2: DINOv2 330MB Download Failing

Problem: PyTorch Hub download kept getting interrupted mid-way, leaving a corrupted partial cache that caused errors on the next run.

Solution: Cleared the torch hub cache directory entirely and re-downloaded from scratch. Verified model loaded without errors before training.

Challenge 3: SegFormer Weights in safetensors Format

Problem: Loading `best_b2.pth` raised an `UnpicklingError` — the file started with `0xFF` which is the safetensors magic byte, not a pickle header.

Solution: Used `weights_only=False` combined with the safetensors loader rather than the default `torch.load` pickle path.

Challenge 4: TypeError in DINOv2 `forward_features`

Problem: Calling `dino_backbone.forward_features(imgs)` raised a `TypeError: NoneType is not iterable` because masks defaulted to `None`.

Solution: Changed the call to pass an explicit list of `None` values matching the batch size: `masks=[None]*batch_size`.

Challenge 5: Sky Class IoU Persistently at 0.17

Problem: DINOv2 consistently scored 0.17 on Sky across all runs. Sky is a large, featureless region — patch-based ViT features do not distinguish it well.

Solution: Ensemble with SegFormer B2 lifted Sky IoU to 0.356. SegFormer's hierarchical encoder handles uniform regions substantially better.

Challenge 6: stringzilla Build Failure on Windows

Problem: `pip install stringzilla` (an albumations dependency) failed on Windows because MSVC 14.0 C++ build tools were not installed.

Solution: Switched to Python 3.11 which had a prebuilt stringzilla wheel available, bypassing the need to compile from source.

7. Conclusion

We set out to build a semantic segmentation system for off-road desert terrain using only synthetic Falcon data, aiming to generalize to unseen environments. After multiple training iterations, our final solution — a weighted ensemble of DINOv2+Head and SegFormer B2 — achieved a mean IoU of 0.6142 on the validation set, with particularly strong results on Road (0.844) and Boundary (0.984).

While working on this project, we learned several practical lessons. Synthetic data, when it includes enough variety in lighting and scene conditions, can train robust computer vision models quite effectively. During experimentation we observed that combining multiple models gave significantly better results — each model compensated for areas where the other struggled. Progressive resizing also turned out to be a surprisingly simple way to push validation mIoU higher without adding more training time.

8. Future Work

Full Backbone Fine-tuning	Partially unfreeze DINOv2 backbone with very low LR — the hail_mary run showed promise but did not complete in time. Could push mIoU above 0.65.
Larger SegFormer	SegFormer B4 or B5 would likely yield better results for difficult classes like Vegetation and Sky.
Domain Adaptation	Explore CycleGAN-style unpaired translation from synthetic to photorealistic desert images for better sim-to-real transfer.
Class-Weighted Loss	Per-class loss weighting proportional to inverse frequency would improve IoU for rare, underrepresented classes.
Real-Time Deployment	Convert ensemble to ONNX/TensorRT and optimize for Jetson Orin for actual UGV deployment.

9. Reproducibility

Requirements: Python 3.11, NVIDIA GPU with CUDA 11.8+. Install dependencies: `py -3.11 -m pip install -r requirements.txt`. If stringzilla fails on Windows, install Microsoft Visual C++ Build Tools first. Then run scripts in order: (1) `train_segmentation_improved.py` → saves `segmentation_head_best.pth`. (2) `train_fast.py` → saves `segmentation_head_fast.pth`. (3) `merge_models.py` → prints per-class IoU and final mIoU. (4) `visualize_ensemble.py` → saves 10 images to `visualization_results/`.

File / Folder	Description
train_segmentation_improved.py	Main DINOv2+Head training script — 30 epochs
train_fast.py	Progressive resizing training script
merge_models.py	Ensemble evaluation — per-class IoU + final mIoU
visualize_ensemble.py	Generates 10 side-by-side comparison images
segmentation_head_best.pth	Best DINOv2+Head checkpoint
segmentation_head_fast.pth	Best progressive-resize checkpoint
best_b2.pth	Best SegFormer B2 checkpoint
visualization_results/	10 comparison PNG images from visualize_ensemble.py
requirements.txt	All Python dependencies
README.md	Step-by-step run instructions